



Building a Material Science Data Intelligence Platform

LLM Intent Extraction · Vector Hybrid Search · Explainable AI

May 2025



Context & Scope



Proof-of-concept

Architectural decisions optimized for demonstrating **rapid prototyping** and **technical reasoning**.



Open-ended Brief

Proactively built "AI-Guru" (**Material Intelligence**) to demonstrate a high-value, practical scientific use case.



Vector Sample Size

ChromaDB embedded a representative sample of the 154k database for execution speed.



Aggressive Caching

Permanent JSON disk cache bypasses LLM delays and saves API credits on pre-computed examples.



Illustrative XAI

Scientific explanations demonstrate pipeline capabilities and are **not verified** conclusions.



AI-Assisted Dev

LLMs used for research and code scaffolding; all architecture and reasoning is original.

```
// DATA_SOURCES: MATERIALS_PROJECT, CHROMADB_SAMPLE  
// VERIFICATION_STATUS: CONCEPTUAL_PROTOTYPE_ONLY
```



Real-World AI Applications in Industry

"AI is actively transforming how industries manage and discover complex data."



Generative Design & Discovery Simulations: Accelerating the discovery of new alloys and drugs by predicting chemical **viability** before physical lab testing.



Digital Twins & Predictive Modeling: Creating virtual representations of physical assets to **simulate** stress and predict maintenance cycles.



Text-to-SQL & Advanced Querying: Empowering business users to query complex relational databases using natural language, without needing SQL.



Semantic Search (RAG): Using Vector Databases and LLMs to retrieve information based on meaning rather than exact keyword matches.

```
// INDUSTRY_TRENDS: GENERATIVE_DESIGN, DIGITAL_TWINS, RAG
// APPLICATION_FOCUS: COMPLEX_DATA_INTELLIGENCE
```



Why We Chose This Path ("AI-Guru")

"Simplifying complex workflows to prove the versatility of modern AI."



The Purpose

Demonstrate AI solving real-world **workflow bottlenecks**. Bridging the gap between human intent and underlying data without requiring complex manual queries.



Showcasing Multiple Technologies

Orchestration of **RAG**, **Semantic Embeddings**, Mathematical Scoring & Ranking, and **Explainable AI (XAI)** working in unison.



The Design Context

Built as an open-ended intelligence engine to synthesize multiple complex capabilities at once, demonstrating scalability beyond narrow problem-solving.

```
// ARCHITECTURE_TYPE: ORCHESTRATED_INTELLIGENCE_ENGINE  
// CORE_VALUE: SYNTHESIS_OF_COMPLEX_CAPABILITIES
```



Problem Statement & AI Vision

"SQL is rigid. Scientific discovery requires semantic intent."



The Problem

The Task 1 database is incredibly powerful, but querying it requires knowing exact **SQL thresholds** (e.g., `band_gap > 1.0 AND energy_above_hull = 0`). This creates technical friction for experts.



The Vision

Allow a material scientist to type: "**Find a cheap, highly stable semiconductor without lithium**" and have the system translate that into mathematical weights and strict elemental filters.



The Goal

Not just to find the material, but to use **Explainable AI (XAI)**:

- Tell the scientist exactly why it was selected
- Potential real-world usage, bridging the gap between raw data and actionable insights.

```
// VISION_ALIGNMENT: SEMANTIC_SEARCH_OVER_SQL  
// DESIGN_GOAL: EXPLAINABLE_SCIENTIFIC_INSIGHTS
```




Deliverables: What Was Planned

- 01 **Data Foundation:** Semantic Feature Engineering Pipeline (Task 1 SQLite → ChromaDB)
- 02 **Retrieval:** Vector Database Integration (Local SentenceTransformers)
- 03 **Orchestration:** LLM Integration & Intent Extraction (**LangChain** + OpenAI)
- 04 **Intelligence:** **Hybrid** Recommendation Engine & Explainable AI Generation
- 05 **Delivery:** Interactive User Interface & Persistent Disk Caching

```
// PIPELINE_STATUS: ALL_DELIVERABLES_COMPLETED  
// TIMEFRAME: MAY_2025_PROJECT_CYCLE
```



Architecture: Hybrid RAG System

"Retrieval-Augmented Generation tailored for Materials Science."

User Query → LLM Intent Extractor → Hybrid Search (Chroma) → Mathematical Re-Ranking → LLM Explainer → UI



1. The Vector Database

- **Model:** `all-MiniLM-L6-v2` (fast, local sentence-transformer).
- **Metadata:** Injected exact numerical values (density, cost, elements) directly into ChromaDB for hard filtering alongside the semantic vector.



2. The Hybrid Search Concept

- **Semantic:** Finds materials that "sound" like the user's need.
- **Deterministic Filter:** Strictly enforces `avoid_elements` (e.g., no Cobalt) at the vector level.
- **Algorithmic Re-ranking:** A custom scoring function that re-ranks results based on mathematical weights (cost vs. stability) extracted by the LLM.

```
// PIPELINE_STAGE: RETRIEVAL_ORCHESTRATION
// SEARCH_METHOD: HYBRID_SEMANTIC_DETERMINISTIC
```



The Semantic Payload: What Did We Embed?

"Empowering semantic search by translating rows into human sentences."

The Semantic Template

"{formula} is a {crystal_system} material with a density of {density} g/cm³, {conductivity}, and is {stability}. It has a composite score of {score} and an element rarity index of {cost_index}."

Example Vectorized Payload:

"LiFeP04 is a orthorhombic material with a density of 3.60 g/cm³, 3.72 eV band gap, and is thermodynamically stable. It has a composite score of 0.84 and an element rarity index of 0.051."

Why does this matter?

By converting raw relational data into natural language, we empower the vector database to perform semantic matching against human queries.

Semantic Match Examples:

- "Find a lightweight cubic material" (Density/Crystal)
- "Highly stable metal" (Stability/Conductivity)
- "Suggest a cheap semiconductor" (Rarity/Band gap)

```
// ETL_PROCESS: NATURAL_LANGUAGE_ENRICHMENT  
// VECTOR_PAYLOAD: SEMANTIC_SENTENCE_EMBEDDING
```




AI Pipeline: Intent & Explainability

"Controlling the LLM with strict schemas."

Phase 1: Intent Extraction

Translates natural language into structured **JSON filters** for the search engine.

- **Weights:**
{conductivity: 0.2, stability: 0.5, density: 0.1, cost: 0.2}
- **Filters:**
{is_metal: False, **avoid_elements**: ["Li", "Mn"]}

Phase 2: Explainable AI (XAI)

Generates human-readable justifications for the retrieved material candidates.

- **Scientific Explanation:**
A detailed breakdown of why the material mathematically fits the user's specific weights.
- **Potential Usage:**
Mapping abstract chemical properties to practical, **real-world engineering applications**.

```
// PIPELINE_PHASE: INTENT_EXTRACTION_AND_JUSTIFICATION  
// CONTROL_TYPE: STRICT_JSON_SCHEMA_ENFORCEMENT
```



Challenges, Trade-offs & Resiliency

"Every shortcut was a deliberate trade-off, not an oversight."

Challenge	Resilient Resolution
Vector Search Bias: Battery datasets are heavily skewed . Filters wiped results.	Massive Over-fetching: Increased top_k pool from 150 to 3,000 before filters.
Unpredictable LLM Formats: Markdown/newlines broke json.loads extraction.	Fail-Safe Parsing: Enforced OpenAI json_object and built a 3-stage fallback parser.

Decision	Chosen For	Trade-off Accepted
Streamlit over FastAPI	Rapid prototyping speed	Not horizontally scalable
Sample 3k materials	Local execution performance	Reduced retrieval coverage
Local all-MiniLM-L6-v2	Zero API costs (local)	Lower semantic accuracy
Persistent JSON cache	Bypassing LLM delays/cost	Not for multi-user access

```
// CHALLENGE_LOG: BIAS_AND_FORMAT_HANDLING  
// SYSTEM_RESILIENCY: MULTI_STAGE_FALLBACK_PARSING
```



Live Demo (Streamlit)

"Experience the pipeline in action: From intent extraction to XAI."

01 Show the **"Intelligent Search"** sidebar

02 Click **"Clear AI Cache"** for live proof

03 Query: **"Find a cheap, highly stable semiconductor without lithium or manganese"**

04 Wait ~5s for live LLM generation

05 Expandable **"AI Understanding"** Block

06 Material Results: Radar charts & properties

07 AI Explainability & **Green Highlight Box**

08 Repeat query for **Instant Disk Cache**

```
// DEMO_MODE: INTERACTIVE_STREAMLIT_WALKTHROUGH
// CACHE_STRATEGY: DISK_PERSISTENT_JSON
```



Final Outcomes & Deliverables

"Ensuring end-to-end reliability from data to user interface."

#	Deliverable	Status
01	Semantic Feature Engineering Pipeline	✓ Completed (SQLite → ChromaDB ETL)
02	Vector Database Integration	✓ Completed (SentenceTransformers)
03	LLM Integration	✓ Completed (LangChain + gpt-4o-mini)
04	Intent Extraction Engine	✓ Completed (JSON Schema Enforced)
05	Hybrid Recommendation Engine	✓ Completed (Semantic + Property re-ranking)
06	Explainable AI (XAI) Generation	✓ Completed (Scientific Explanations)

```
// PIPELINE_STATUS: ALL_MODULES_OPERATIONAL
// SYSTEM_OUTPUT: VERIFIED_END_TO_END
```



Thank You



Appendix — Discussion Ready

"Supplementary materials for deep-dive sessions."



Appendix A — The Mathematical Re-ranking Equation

"Hybrid retrieval: Semantic search for candidates, deterministic logic for ranking."

ChromaDB's semantic vector search handles *initial retrieval*. Once candidates are returned, the ranking is done entirely by a deterministic property score — the vector distance is **not** part of the final ranking equation. This is what makes it a **hybrid** system.

```
# Step 1: Normalise each property score to 0-1 range
cond_score = max(0, 1.0 - abs(band_gap - 1.5) / 17.89)
stab_score = metadata["stability_score"]
dens_score = max(0, 1.0 - (density / 26.58))
cost_score = max(0, 1.0 - relative_cost_index)

# Step 2: Apply LLM-extracted weights
score = (w_cond * cond_score + w_stab * stab_score +
         w_dens * dens_score + w_cost * cost_score)
```

The semantic vector is used only for **candidate pool retrieval**. **Final ranking** is purely **property-driven**, giving full scientific interpretability.

```
// LOGIC_TYPE: HYBRID_DETERMINISTIC_RANKING
// PARAMETERS: BAND_GAP, STABILITY, DENSITY, COST
```



Appendix B — Intent Extraction Prompt (LLM)

"Translation layer: Converting natural language into machine-readable JSON constraints."

This is the exact system prompt sent to **gpt-4o-mini** (configurable via OPENAI_MODEL_NAME env var) to translate user queries into strict JSON.

```
# System Instruction Set
```

```
You are a materials science AI assistant. Extract search parameters from the query.
```

```
Output MUST be valid JSON matching this schema:
```

```
{  
  "weights": {"conductivity": float, "stability": float, "density": float, "cost": float},  
  "filters": {"is_metal": bool|null, "avoid_elements": [list]}  
}
```

```
# Constraints
```

- Weights must sum to 1.0.
- If they ask for cheap, give high weight to cost.
- If they ask to avoid an element, put its SYMBOL in avoid_elements.

```
// AGENT_ROLE: INTENT_EXTRACTION_ENGINE  
// SCHEMA_STRICTNESS: VALIDATED_JSON_ONLY
```



Appendix C — Double-Fallback JSON Parser

"Resilient parsing logic to guarantee orchestration pipeline stability."

Because LLMs occasionally output malformed JSON (like wrapping it in markdown code blocks), we built **resilient parsing** logic to guarantee the orchestration pipeline never breaks.

```
# From src/ai/llm_chain.py - Intent Extraction Fallback
def extract_intent(self, query: str) -> dict:
    prompt = self.intent_prompt.format(query=query)
    content = self.llm.invoke(prompt).content.strip()

    # 1. Clean markdown formatting
    if content.startswith("`json`"): content = content[7:-3]
    elif content.startswith("`"): content = content[3:-3]

    try: return json.loads(content) # 2. Strict parse
    except Exception as e:

    # 3. Ultimate Fallback: Safe defaults
    return { "weights": {"conductivity": 0.25, "stability": 0.25, ...},
            "filters": {"is_metal": None, "avoid_elements": []} }
```

```
// PARSE_TYPE: REGULAR_EXPRESSION_STRIPPER
// ERROR_HANDLING: NON_BLOCKING_DEFAULTS
```



Appendix D — Permanent Disk Caching Architecture

"Streamlit's `@st.cache_data` lives in RAM and dies on Docker restarts. We built a permanent JSON cache to save OpenAI API costs completely."

By mounting a persistent Docker volume, we ensure that **expensive LLM calls** are cached across container lifecycles, reducing latency from ~5s to near-zero for repeated queries.

```
CACHE_FILE = "/app/data/ai_cache.json" # Mounted via Docker volume
def run_cached_query(workflow, q: str):
    # 1. Check permanent disk cache
    if os.path.exists(CACHE_FILE):
        with open(CACHE_FILE, "r") as f: cache = json.load(f)
        if q in cache: return cache[q]["explanation"], cache[q]["candidates"]
    # 2. Live LLM Generation (~5 seconds)
    explanation, candidates, intent = workflow.process_query(q)
    # 3. Write securely to disk
    cache[q] = {"explanation": exp, "candidates": cand, "intent": intent}
    with open(CACHE_FILE, "w") as f: json.dump(cache, f)
```

```
// CACHE_STRATEGY: PERSISTENT_JSON_DISK_STORE
// COST_OPTIMIZATION: ZERO_LATENCY_REPEATS
```




Appendix E — Production Roadmap (To-Do)

"From Prototype to Enterprise-Grade: A paradigm shift in architecture and governance."

⚠ *Important Disclaimer: This list is not comprehensive; a detailed migration plan requires proper requirements analysis. Trade-offs in this PoC were deliberate; production needs may fundamentally change the architecture.*

Component	Proof-of-Concept	Production Equivalent
Vector Search	Local ChromaDB	Pinecone / AWS OpenSearch
Orchestration	Synchronous LangChain	Async Celery + Redis
Web Framework	Streamlit	React + FastAPI
Embeddings	Local all-MiniLM	Fine-tuned domain models

1. Guardrails

NeMo Guardrails for safety. Strict persona to block non-science queries.

2. Security

SSO integration (Okta/AD). Row-level security for data lineage.

3. Observability

LangSmith for latency. RAG metrics (TruLens) for accuracy.

4. UI/Feedback

Async streaming (SSE). Integrated user feedback loops.

```
// ROADMAP_STAGE: ENTERPRISE_MIGRATION_PLANNING
// PRIORITY: SECURITY_AND_SCALABILITY
```